hdoc day 39 by trishil chauhan

C Programming - Variadic Functions and Command-Line Arguments

Program 1: Weighted average using stdarg.h

Aim: Accept a count followed by value-weight pairs, reject invalid weights and zero total weight, and calculate the weighted average.

```
#include <stdio.h>
#include <stdarg.h>
#include <math.h>

/* Each pair is passed as two double arguments. */
double weightedAverage(int count, ...) {
    va_list args;
    double weightedSum = 0.0, totalWeight = 0.0;
    int invalid = 0;

    if (count <= 0) {
        fprintf(stderr, "Error: count must be positive.\n");
        return NAN;
    }

    va_start(args, count);
    for (int i = 0; i < count; i++) {
        double value = va_arg(args, double);
        double weight = va_arg(args, double);
        if (!isfinite(value) || !isfinite(weight) || weight < 0.0)
            invalid = 1;
        else {
            weightedSum += value * weight;
            totalWeight += weight;
        }
    }
    va_end(args);

    if (invalid) {
        fprintf(stderr, "Error: invalid value or negative weight.\n");
        return NAN;
    }
    if (totalWeight == 0.0) {
        fprintf(stderr, "Error: total weight is zero.\n");
        return NAN;
    }
    return weightedSum / totalWeight;
}

int main(void) {
    double result = weightedAverage(3, 80.0, 0.3,
                                    75.0, 0.2, 90.0, 0.5);
    if (!isnan(result))
        printf("Weighted average: %.2f\n", result);
    return 0;
}
```

Sample output

```
Weighted average: 84.00
```

Calculation: (80 x 0.3 + 75 x 0.2 + 90 x 0.5) / (0.3 + 0.2 + 0.5) = 84.00

Note: Variadic arguments must be supplied as double values. NAN signals an invalid result; the function also prints an error message.

Program 2: Statistics from command-line integers

Aim: Validate integer arguments, ignore invalid entries, sort valid values, and display minimum, maximum, average, median, and second-largest distinct value.

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>
#include <string.h>

int parseInteger(const char *s, int *out) {
    char *end;
    long value;
    if (*s == '\0') return 0;
    errno = 0;
    value = strtol(s, &end, 10);
    if (errno == ERANGE || *end != '\0' ||
        value < INT_MIN || value > INT_MAX) return 0;
    *out = (int)value;
    return 1;
}

int compareInts(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}

void sortIntegers(int a[], int n) {
    qsort(a, (size_t)n, sizeof(a[0]), compareInts);
}

double calculateAverage(const int a[], int n) {
    long double sum = 0;
    for (int i = 0; i < n; i++) sum += a[i];
    return (double)(sum / n);
}

double calculateMedian(const int a[], int n) {
    if (n % 2) return a[n / 2];
    return ((double)a[n / 2 - 1] + a[n / 2]) / 2.0;
}

int secondLargestDistinct(const int a[], int n, int *result) {
    for (int i = n - 2; i >= 0; i--) {
        if (a[i] != a[n - 1]) {
            *result = a[i];
            return 1;
        }
    }
    return 0;
}

int main(int argc, char *argv[]) {
    int *a = malloc((size_t)(argc > 1 ? argc - 1 : 1) * sizeof(int));
    int n = 0, second;
    if (!a) {
        fprintf(stderr, "Error: memory allocation failed.\n");
        return 1;
    }
    for (int i = 1; i < argc; i++) {
        int value;
        if (parseInteger(argv[i], &value)) a[n++] = value;
        else printf("Invalid argument ignored: %s\n", argv[i]);
    }
    if (n == 0) {
        puts("Error: no valid integers supplied.");
        free(a);
        return 1;
```

```
    }
    sortIntegers(a, n);
    printf("Valid integers in ascending order:");
    for (int i = 0; i < n; i++) printf(" %d", a[i]);
    putchar('\n');
    printf("Minimum: %d\nMaximum: %d\n", a[0], a[n - 1]);
    printf("Average: %.2f\n", calculateAverage(a, n));
    printf("Median: %.2f\n", calculateMedian(a, n));
    if (secondLargestDistinct(a, n, &second))
        printf("Second-largest distinct value: %d\n", second);
    else puts("Second-largest distinct value: Not available");
    free(a);
    return 0;
}
```

Compile and run

```
cc program.c -o program
./program 12 5 abc 8 12 20 3
```

Sample output

```
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
```

Result

Both programs implement the required operations with input validation and appropriate user-defined functions.